AICW ML COMPANION

Understanding Machine Learning

A plain-English guide, with every idea explained twice: once in words, once with real numbers

Chapter 3 · Read alongside the lecture slides · Every worked example uses real numbers, start to finish

How to use this companion. This book expands the lecture slides into full explanations, in plain words. Five coloured boxes guide you. The blue box gives the idea in one breath. The amber box gives a real-life picture of it. The green box solves a real problem, step by step, with numbers. The red box names the mistake beginners make. The purple box is the one line to remember. Read them in that order and the maths never jumps ahead of you.

1 What is machine learning?

Your email inbox quietly sorts spam from real mail. Nobody wrote a rule for every possible spam message — that list would never end. Instead, the program looked at thousands of past emails and learned the pattern. That is machine learning: a computer program that gets better at a job by seeing examples, not by being told every exact step.

☞ The intuition

Machine learning is not magic and not "the computer thinking". It is a program that adjusts itself using data, so its answers get closer to correct the more examples it sees.

The first widely used formal definition comes from computer scientist Tom M. Mitchell (1997):

> *"A computer program is said to* learn *from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."*

Three letters carry the whole idea. E is the **experience** — the past examples the program studies. T is the **task** — the job it is being trained to do. P is the **performance measure** — the number that tells us if it is doing that job well.

★ Everyday picture

Think of a child learning to ride a bicycle. The *experience* E is every wobbly attempt in the park. The *task* T is "ride the bike without falling". The *performance measure* P is how far they get before putting a foot down. Nobody hands them a rulebook of balance equations — they improve purely from practice, exactly like a learning algorithm improves from data.

Piece	Spam filtering	Handwritten digit recognition
E (experience)	Thousands of emails already labelled spam or not spam	Thousands of digit images already labelled 0–9
T (task)	Decide if a new email is spam	Decide which digit (0–9) an image shows
P (performance)	% of emails correctly labelled	% of digits correctly identified

Where this shows up in ML. Every model you will meet in this chapter — a spam filter, a house-price predictor, a digit reader — is just a different choice of T and a different shape for the program that learns from E and is scored by P.

✗ Watch out

Don't confuse "the program improves" with "the program understands". A spam filter does not know what spam *means*; it only knows which word patterns showed up with the label "spam" before.

✔ Key takeaway

Machine learning = a program that gets better at a task T by studying experience E, scored by P.

2 Three ways a machine can learn

Not every learning problem hands you the same kind of help. Sometimes you have an answer key. Sometimes you don't. Sometimes the only feedback is "good move" or "bad move" after the fact.

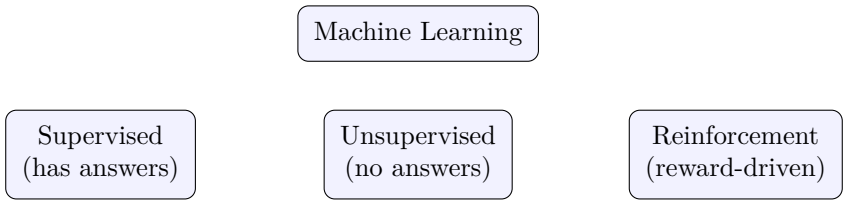
☞ The intuition

Supervised learning: you have labelled examples (input *and* the correct answer) — learning with a teacher. **Unsupervised learning**: you only have inputs, no answers — the program finds structure on its own. **Reinforcement learning**: an agent takes actions and learns from rewards and penalties, by trial and error.

★ Everyday picture

Supervised is studying with flashcards that have the answer printed on the back. **Unsupervised** is sorting a mixed laundry basket into piles of "things that look alike", with nobody telling you the pile names. **Reinforcement** is training a dog: no manual, just a treat when it does the right thing and nothing when it doesn't.

Type	What it gets	Goal	Example
Supervised	Inputs *with* correct answers	Learn the input→answer mapping	Spam filter, price predictor
Unsupervised	Inputs only, no answers	Find hidden groups or structure	Customer segmentation
Reinforcement	Actions + rewards/penalties	Maximise total reward over time	Game-playing AI



How to read it: the same starting point, machine learning, branches into three families depending on what kind of feedback the program is given.

Where this shows up in ML. A large language model is trained with a mix of all three ideas in sequence: it first learns raw patterns of language in an unsupervised way, is then fine-tuned on labelled supervised examples, and can be further polished with reinforcement learning from human feedback.

✗ Watch out

Real projects rarely fit one box perfectly. Many production systems blend approaches — for example, using a little labelled data plus a lot of unlabelled data (semi-supervised learning).

✔ Key takeaway

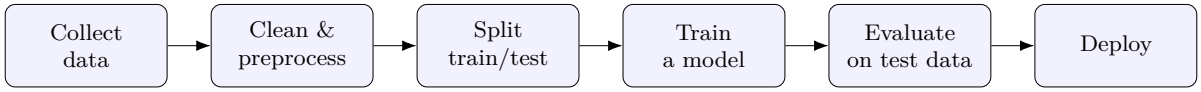
Ask first: "Do I have the correct answers for my data?" That single question tells you whether you need supervised or unsupervised learning.

3 The shape of a machine learning project

Before any clever algorithm runs, most of the real work is getting the data ready and setting up a fair test.

☞ The intuition

Every supervised learning project follows the same six-stage shape: collect data, clean it, split it into a training part and a testing part, train a model on the training part, check it on the testing part, then ship it.



How to read it: data flows left to right through six stages. If evaluation looks bad, you loop back — usually to preprocessing or model choice — not straight to deployment.

3.1 Data preprocessing

Raw data is messy: missing values, text where you need numbers, values on wildly different scales (age in years, income in hundreds of thousands). **Data preprocessing** means cleaning and reshaping raw data so a model can use it. It is the very first step, and skipping it quietly wrecks everything that follows.

3.2 Splitting into a training set and a test set

We never judge a model on the data it studied. We hide part of the data — the **test set** — until training is completely finished, then use it as a fair, unseen exam.

✎ Worked example: splitting a 100-row dataset

A dataset has 100 rows. We want 80% for training and 20% held back for testing.

Step 1. Write down what we know. Total rows = 100. Test fraction = 0.2 (that is what `test_size=0.2` means in code).

Step 2. Compute the test-set size. $100 \times 0.2 = 20$ rows go to the test set.

Step 3. Compute the training-set size. $100 - 20 = 80$ rows go to the training set.

Step 4. Fix the randomness. Setting `random_state=42` means the "random" split is the same every time you re-run the code — useful so your results are reproducible.

So: 80 rows to learn from, 20 rows kept completely unseen for the final check.

```
from sklearn.model_selection import train_test_split

# X holds the input features, y holds the target (the answer column)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Where this shows up in ML. Every benchmark you read about — "model X scores 94% on ImageNet" — is reporting a score on a held-out test set the model never trained on. Without that split, a reported accuracy is meaningless.

✗ Watch out

Testing a model on the same rows it trained on is like grading a student on the exact quiz they memorised the answers to. The score will look great and mean nothing.

✔ Key takeaway

Clean the data, then split it *before* training, and never let the model see the test rows until the final check.

4 Regression or classification?

Predicting a student's exact exam score is a different kind of problem from predicting whether a patient has a disease.

☞ The intuition

Regression predicts a number that can take any value along a scale (a score, a price, a temperature). **Classification** predicts a category from a fixed, known list (spam/not-spam, cancer/no-cancer).

★ Everyday picture

Regression is guessing someone's exact age from a photograph — any number is possible, 27, 41.5, whatever. Classification is guessing whether they are a child, teenager, or adult — three fixed boxes, no in-between number.

Scenario	Output being predicted	Type
Predict a student's final exam score from study hours, attendance, and assignment scores	A number, e.g. 76.4 out of 100	Regression
Decide whether a patient has cancer from symptoms and history	One of two categories: cancer / no cancer	Classification

✗ Watch out

A category can be *written* as a number (0 = no cancer, 1 = cancer) without becoming a regression problem. Ask "can the true answer be any value in between?" — if no, it's classification.

✔ Key takeaway

Predicting an *amount* ⇒ regression. Predicting a *label from a fixed list* ⇒ classification.

5 Linear regression

Linear regression answers: "given the trend so far, what number comes next?" It draws the single straight line that best follows a set of points, then reads predictions off that line.

☞ The intuition

Linear regression finds the straight line $y = b_0 + b_1 x$ that stays as close as possible, on average, to every data point. y is the **dependent variable** (what we're predicting); x is the **independent variable** (what we use to predict it).

★ Everyday picture

A parent tracks a child's height every year on the wall. The dots roughly climb in a straight line. Without doing any formal maths, a parent can eyeball that line and guess next year's height. Linear regression does that eyeballing with exact arithmetic instead of a guess.

5.1 Finding the best-fit line

The **best-fit line** is the one line that makes the total squared distance between the line and every point as small as possible — that total is called the **sum of squared errors**. For one input variable, two formulas give the line directly:

$$b_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}, \qquad b_0 = \bar{y} - b_1 \bar{x}$$

Here x_i, y_i are the individual data points, $\bar{x}$ and $\bar{y}$ are their averages (means), b_1 is the **slope** (how much y changes per unit of x), and b_0 is the **intercept** (the predicted y when $x = 0$).

✎ Worked example: predicting an exam score from study hours

Five students studied for 1, 2, 3, 4, and 5 hours, and scored 50, 55, 65, 70, and 80 out of 100. Find the best-fit line and predict each score.

Step 1. Write down what we know. $x = (1, 2, 3, 4, 5)$, $y = (50, 55, 65, 70, 80)$, $n = 5$.

Step 2. Find the averages. $\bar{x} = 15/5 = 3$. $\bar{y} = 320/5 = 64$.

Step 3. Find each deviation from the average.

$$x_i - \bar{x} = (-2, -1, 0, 1, 2), \qquad y_i - \bar{y} = (-14, -9, 1, 6, 16)$$

Step 4. Multiply the deviations and add them up.

$$\sum (x_i - \bar{x})(y_i - \bar{y}) = 28 + 9 + 0 + 6 + 32 = 75$$

Step 5. Square the x deviations and add them up.

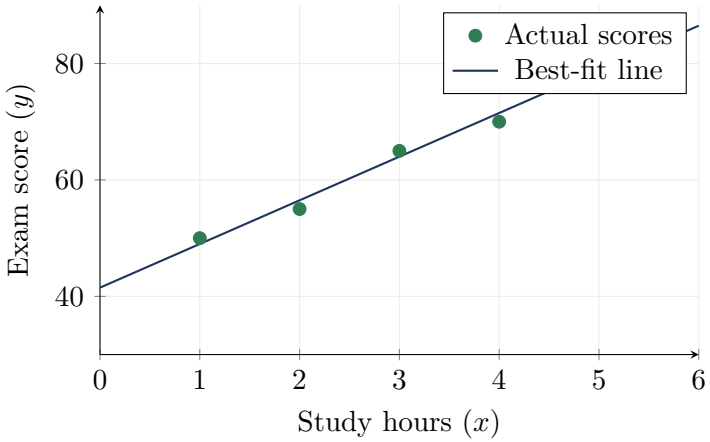
$$\sum (x_i - \bar{x})^2 = 4 + 1 + 0 + 1 + 4 = 10$$

Step 6. Compute the slope. $b_1 = 75/10 = 7.5$.

Step 7. Compute the intercept. $b_0 = 64 - 7.5(3) = 64 - 22.5 = 41.5$.

Step 8. Write the line and predict. $\hat{y} = 41.5 + 7.5x$. For $x = 1$: $\hat{y} = 49$; $x = 2$: 56.5; $x = 3$: 64; $x = 4$: 71.5; $x = 5$: 79.

So each extra hour of study is worth **7.5 points**, on this small sample.



How to read it: the dots are the real students; the navy line is the single straight line that sits closest to all five dots at once — the line the worked example computed.

5.2 How good is the fit? Three error scores

Residual = actual value − predicted value, the leftover gap the line missed. Three standard scores summarise how big the residuals are, on average:

$$\text{MAE} = \frac{1}{n}\sum |y_i - \hat{y}_i|, \qquad \text{MSE} = \frac{1}{n}\sum (y_i - \hat{y}_i)^2, \qquad \text{RMSE} = \sqrt{\text{MSE}}$$

MAE (mean absolute error) treats every mistake the same size. MSE (mean squared error) punishes big mistakes much harder, because it squares them. RMSE (root mean squared error) undoes that squaring, so it is back in the same units as y (points, not points-squared).

✎ **Worked example: scoring the exam-score line**

Using the residuals from the line above: $1, -1.5, 1, -1.5, 1$.

Step 1. Take absolute values for MAE. $1, 1.5, 1, 1.5, 1$. Sum $= 6$. MAE $= 6/5 = 1.2$.

Step 2. Square each residual for MSE. $1, 2.25, 1, 2.25, 1$. Sum $= 7.5$. MSE $= 7.5/5 = 1.5$.

Step 3. Take the square root for RMSE. RMSE $= \sqrt{1.5} \approx 1.2247$.

On average, this line's predictions are off by about **1.2 points** out of 100 — a very tight fit.

Where this shows up in ML. A linear layer at the end of a neural network doing regression (say, predicting house price) is literally this same idea, just with many more input variables at once. Linear regression is also the standard "sanity check" baseline before trying anything fancier.

✗ Watch out

Linear regression assumes the true relationship is roughly a straight line. Fit it to strongly curved data or data with wild outliers, and it will confidently draw the wrong line. Always plot the data first.

✔ Key takeaway

The best-fit line minimises total squared error. MAE, MSE, and RMSE all measure how far off that line's predictions are — smaller is always better.

6 Logistic regression

What if the answer isn't a number, but a yes or no?

☞ The intuition

Logistic regression turns a weighted sum of the inputs into a **probability** between 0 and 1, using the **sigmoid function**. If that probability is 0.5 or higher, predict class 1; otherwise, predict class 0.

★ Everyday picture

Imagine weighing evidence for a court verdict: each clue adds or subtracts confidence. Logistic regression adds up weighted clues into one confidence score, then squeezes that score into "how likely is guilty", between 0% and 100%, before making the final call.

First compute a weighted sum, exactly like linear regression: $z = b_0 + b_1 x_1 + b_2 x_2 + \dots$. Then pass z through the sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Here $e \approx 2.71828$ is Euler's number, and $\sigma(z)$ (the Greek letter sigma) always lands strictly between 0 and 1, no matter how large or small z is.

✎ Worked example: classifying two emails by their score z

Step 1. Email A has $z = 0.8$. Compute $e^{-0.8} \approx 0.4493$.

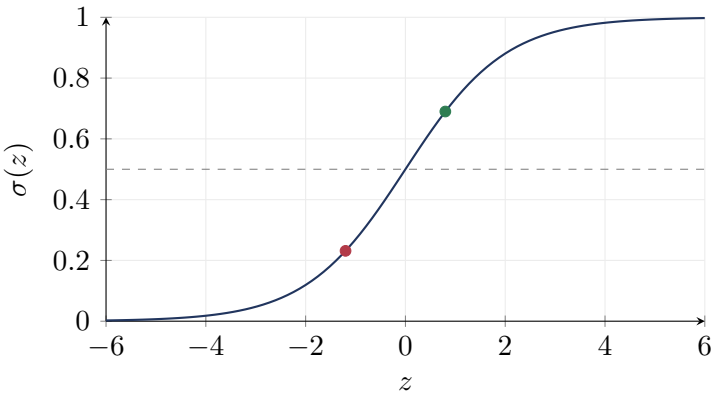
Step 2. Finish the sigmoid. $\sigma(0.8) = 1/(1 + 0.4493) = 1/1.4493 \approx 0.6900$.

Step 3. Classify. $0.6900 \geq 0.5$, so Email A is predicted **spam** (class 1), with 69% confidence.

Step 4. Email B has $z = -1.2$. Compute $e^{1.2} \approx 3.3201$.

Step 5. Finish the sigmoid. $\sigma(-1.2) = 1/(1 + 3.3201) = 1/4.3201 \approx 0.2315$.

Step 6. Classify. $0.2315 < 0.5$, so Email B is predicted **not spam** (class 0).



How to read it: the S-shaped curve is the sigmoid. Email A (green dot) lands above the 0.5 line, so it is classified spam; Email B (red dot) lands below it.

	Linear regression	**Logistic regression**
Predicts	A continuous number	A probability (0 to 1), then a class
Output shape	A straight line	An S-shaped curve
Used for	Regression	Binary classification

Where this shows up in ML. The sigmoid is exactly the activation function on the output neuron of a binary-classification neural network. Its multi-class cousin, softmax, does the same squashing job across many classes at once (as used in image classifiers and language models).

✗ Watch out

Logistic regression assumes a roughly straight-line relationship between the inputs and the log-odds of the outcome. Highly tangled, non-linear boundaries need a more flexible model.

✔ Key takeaway

Logistic regression = weighted sum → sigmoid → probability → threshold at 0.5 → class.

7 Classification metrics

"85% accuracy" sounds great — until you learn that 95% of the emails in the dataset were not spam anyway, and the model just guessed "not spam" every single time.

☞ The intuition

For classification we count four outcomes in a **confusion matrix**: how often the model was right about each class, and how often it was wrong about each class.

★ Everyday picture

Airport security screening: a **true positive** correctly flags an actual threat. A **false positive** wrongly flags an innocent traveller. A **false negative** misses a real threat. A **true negative** correctly waves through a safe traveller. The last one is the most common and the least talked about — exactly like "not spam" emails.

	Predicted spam	**Predicted not spam**
Actually spam	True Positive (TP)	False Negative (FN)
Actually not spam	False Positive (FP)	True Negative (TN)

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}, \quad \text{Precision} = \frac{TP}{TP+FP}, \quad \text{Recall} = \frac{TP}{TP+FN}, \quad \text{F1} = \frac{2 \cdot P \cdot R}{P+R}$$

Precision asks: "of everything I flagged as spam, how much really was spam?" **Recall** asks: "of all the real spam out there, how much did I actually catch?" **F1** is a single number that balances both.

✎ **Worked example: scoring a spam filter on 100 emails**

The filter checked 100 emails: TP $= 40$, FP $= 10$, FN $= 5$, TN $= 45$.

Step 1. Accuracy. $(40+45)/100 = 85/100 = 0.85$, or **85%**.

Step 2. Precision. $40/(40+10) = 40/50 = 0.80$, or **80%**.

Step 3. Recall. $40/(40+5) = 40/45 \approx 0.8889$, or **88.9%**.

Step 4. F1 score. $2(0.80)(0.8889)/(0.80+0.8889) = 1.4222/1.6889 \approx 0.8421$, or **84.2%**.

This filter is slightly better at *catching* spam (recall) than at *never crying wolf* on real mail (precision).

Where this shows up in ML. Real systems tune their decision threshold for the metric that matches the cost of a mistake — a cancer-screening model is tuned for very high recall (never miss a real case), even if that means more false alarms (lower precision).

✗ **Watch out**

On imbalanced data, accuracy can look excellent while the model is nearly useless. Always check precision and recall as well, especially when one class is rare.

✔ **Key takeaway**

Choose the metric that matches what a mistake actually costs you — not just the easiest number to report.

8 Decision trees

How would you explain a loan-approval decision out loud to a customer? A sequence of yes/no questions is easy to say and easy to trust.

☞ **The intuition**

A **decision tree** asks a series of yes/no (or threshold) questions about the data, splitting it into smaller and smaller, purer and purer groups, until it reaches a final answer at a **leaf**.

★ **Everyday picture**

It's the game of "20 Questions". A good player asks the question that splits the remaining possibilities most evenly first ("Is it alive?"), not a useless one ("Is it blue?" when almost nothing is blue). A decision tree always asks the most useful question first, at every step.

To pick the "most useful" question, a tree measures how mixed up a group is using **entropy**:

$$H(S) = -\sum_i p_i \log_2 p_i$$

S is the current group of examples, p_i is the fraction of group S belonging to class i, and $\log_2$ is logarithm base 2 (entropy is measured in **bits**). Entropy is 0 when a group is perfectly pure (all one class) and largest when a group is split evenly.

Information gain is how much a question reduces entropy: entropy before the split, minus the weighted average entropy after the split. The tree always picks the question with the highest information gain.

✎ **Worked example: choosing the best first question**

14 past days are recorded: on 9 of them people played outside, on 5 they didn't. One feature is *Outlook*, which splits the 14 days into Sunny (2 played, 3 didn't), Overcast (4 played, 0 didn't), and Rain (3 played, 2 didn't).

Step 1. Entropy before any split. $p_{\text{yes}} = 9/14 = 0.643$, $p_{\text{no}} = 5/14 = 0.357$.

$$H(S) = -0.643 \log_2 0.643 - 0.357 \log_2 0.357 \approx 0.410 + 0.530 = 0.940 \text{ bits}$$

Step 2. Entropy of the Sunny group (2 yes, 3 no).

$$H(\text{Sunny}) = -0.4 \log_2 0.4 - 0.6 \log_2 0.6 \approx 0.529 + 0.442 = 0.971 \text{ bits}$$

Step 3. Entropy of the Overcast group (4 yes, 0 no). All one class, so $H(\text{Overcast}) = 0$.

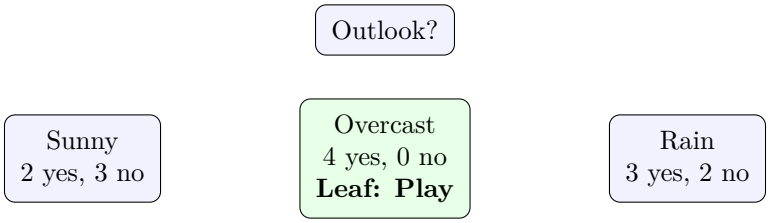
Step 4. Entropy of the Rain group (3 yes, 2 no). By the same maths as Sunny (with the roles swapped): $H(\text{Rain}) \approx 0.971$ bits.

Step 5. Weighted average entropy after the split.

$$\tfrac{5}{14}(0.971) + \tfrac{4}{14}(0) + \tfrac{5}{14}(0.971) = 0.714 \times 0.971 \approx 0.694$$

Step 6. Information gain. $0.940 - 0.694 = \mathbf{0.246}$ bits.

Asking about *Outlook* first removes about a quarter of the "mixed-upness" in one question — a strong first split.



How to read it: the root question splits 14 days into three branches. Overcast is already pure (entropy 0), so it becomes an immediate leaf; Sunny and Rain still need another question.

Where this shows up in ML. Decision trees are the building block of Random Forests and gradient boosted trees (XGBoost, LightGBM) — the workhorse models behind most tabular-data competitions and credit scoring systems.

✗ **Watch out**

A tree grown too deep will keep splitting until each leaf holds a single training example — perfect on training data, and often terrible on new data. This is **overfitting**; limiting tree

depth (pruning) prevents it.

✔ Key takeaway

A decision tree always asks the question that cuts entropy the most, one step at a time, until each group is pure enough to answer.

9 Random forest

One opinion can be wrong. The average of a hundred independent opinions is usually far more reliable.

☞ The intuition

A **random forest** trains many decision trees, each on a random sample of the data and a random subset of features, then combines their answers by majority vote (classification) or averaging (regression).

★ Everyday picture

Instead of trusting one doctor's diagnosis, you ask a hundred doctors, each of whom only saw part of your test results, and go with whatever most of them agree on. Individual mistakes cancel out; the shared signal survives.

✎ Worked example: combining five trees' votes

Five trees each vote on whether a loan applicant should be approved: Yes, No, Yes, Yes, No.

Step 1. Count each vote. Yes: 3 votes. No: 2 votes.

Step 2. Take the majority. 3 out of 5 said Yes, so the forest's final decision is **Approve**.

Where this shows up in ML. Random forests remain a strong, fast baseline for structured/tabular data — spreadsheets of numbers and categories — even sitting next to deep learning; widely used in fraud detection and credit risk.

✗ Watch out

More trees generally help, but with diminishing returns and rising compute cost. Too few trees make the "vote" unstable and easily swayed by one unusual tree.

✔ Key takeaway

A random forest trades the fragility of one tree for the stability of many slightly different trees, voting together.

10 Naive Bayes classifier

How does a spam filter guess your email's intent from just a few words like "free" or "prize"?

☞ The intuition

Naive Bayes starts with a **prior belief** about how likely each class is, then updates that belief using each clue in the data, using Bayes' theorem.

Bayes' theorem describes how to update a belief given new evidence:

$$P(A \mid B) = \frac{P(B \mid A)\,P(A)}{P(B)}$$

A is the hypothesis we care about (e.g. "this email is spam"); B is the evidence we observe (e.g. "the word 'free' appears"). $P(A)$ is the **prior** — how likely A was before seeing any evidence. $P(B \mid A)$ is the **likelihood** — how likely the evidence is, if A is true. $P(A \mid B)$ is the **posterior** — the updated belief after seeing the evidence.

★ Everyday picture

A detective starts with a hunch about a suspect (the prior). Each new clue either strengthens or weakens that hunch (the likelihood), producing an updated suspicion level (the posterior). Naive Bayes is that exact update, done with numbers instead of gut feeling.

The "naive" part: the model assumes every clue (every word, in a spam filter) is independent of every other clue, given the class. That's rarely exactly true — but it works well in practice and is very fast to compute.

✎ Worked example: is an email containing "free" spam?

From past data: $P(\text{spam}) = 0.4$, $P(\text{not spam}) = 0.6$. Among spam emails, 50% contain the word "free": $P(\text{free} \mid \text{spam}) = 0.5$. Among real emails, only 10% do: $P(\text{free} \mid \text{not spam}) = 0.1$.

Step 1. Score the "spam" hypothesis. $P(\text{free} \mid \text{spam})\,P(\text{spam}) = 0.5 \times 0.4 = 0.20$.

Step 2. Score the "not spam" hypothesis. $P(\text{free} \mid \text{not spam})\,P(\text{not spam}) = 0.1 \times 0.6 = 0.06$.

Step 3. Normalise so the two scores sum to 1. Total $= 0.20 + 0.06 = 0.26$.

Step 4. Compute each posterior. $P(\text{spam} \mid \text{free}) = 0.20/0.26 \approx 0.7692$; $P(\text{not spam} \mid \text{free}) = 0.06/0.26 \approx 0.2308$.

Step 5. Classify. $0.7692 > 0.2308$, so the email is classified **spam**, with about **77%** confidence.

Where this shows up in ML. Naive Bayes is a classic, fast baseline for text classification — spam filters and sentiment analysis — because it needs very little training data and is cheap to compute even today.

✗ Watch out

If a word never once appeared with a class during training, its likelihood is exactly 0 — and multiplying by 0 wipes out the whole score, no matter how strong the other clues are. This is fixed with **smoothing**: adding a small fake count to every word so no probability is ever exactly zero.

✔ Key takeaway

Naive Bayes multiplies the prior by each independent clue's likelihood, normalises, and picks the class with the highest score.

11 Support vector machine (SVM)

Given two clouds of dots, what is the most confident possible way to draw a line between them?

☞ The intuition

An SVM finds the dividing line (or, in higher dimensions, a **hyperplane**) that leaves the biggest possible gap — the **margin** — between the two classes.

★ Everyday picture

Imagine building a fence between two rival neighbourhoods. The fairest place to build it is exactly halfway between the two nearest houses on either side — as far as possible from both. An SVM builds that same "fairest" fence between two classes of data.

A hyperplane is written $w \cdot x + b = 0$, where w is a vector of weights (it points perpendicular to the dividing line) and b is a bias term shifting the line. The **support vectors** are the points closest to the line — they are the only points that decide where the line goes. The distance from any point x to the hyperplane is:

$$\text{distance} = \frac{|w \cdot x + b|}{\|w\|}, \qquad \text{margin} = \frac{2}{\|w\|}$$

$\|w\|$ (read "norm of w") is the length of the weight vector: $\|w\| = \sqrt{w_1^2 + w_2^2}$ for two dimensions.

✎ Worked example: classifying a new point

Suppose training gave the hyperplane $w = (2, 1)$, $b = -4$ (that is, $2x_1 + x_2 - 4 = 0$). Classify the point $(3, 2)$.

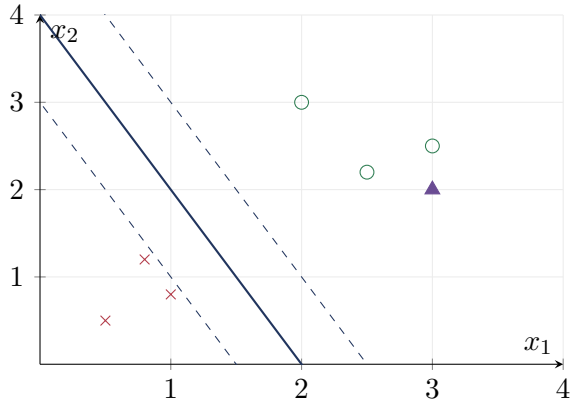
Step 1. Compute $\|w\|$. $\|w\| = \sqrt{2^2 + 1^2} = \sqrt{5} \approx 2.2361$.

Step 2. Plug the point into $w \cdot x + b$. $2(3) + 1(2) - 4 = 6 + 2 - 4 = 4$.

Step 3. Compute the signed distance. $4/2.2361 \approx 1.7889$. It is positive, so the point sits on the "positive" side of the fence.

Step 4. Compute the margin width, for reference. $2/2.2361 \approx 0.8944$ — the gap the SVM guaranteed between the two closest points of each class.

The point $(3, 2)$ is classified as **class** $+1$, comfortably inside its own territory.



How to read it: circles and crosses are the two classes; the solid line is the dividing hyperplane; the two dashed lines mark the margin edges. The purple triangle is the test point from the worked example, sitting inside the circle class's territory.

Where this shows up in ML. SVMs were the state of the art for text classification and small structured datasets before deep learning became dominant. The "kernel trick" that lets SVMs bend curved boundaries is a close cousin of the similarity scoring used in modern attention mechanisms.

✗ Watch out

SVMs measure distance, so features on very different scales (e.g. age in years vs. income in rupees) will silently dominate the result unless every feature is scaled first.

✔ Key takeaway

An SVM doesn't just separate two classes — it finds the widest possible safety margin between them.

12 Unsupervised learning and clustering

How does a streaming service group "similar" shows without anyone labelling genres by hand?

☞ The intuition

Unsupervised learning looks for structure in data that has no labels. **Clustering** is the most common form: grouping data points so that points inside a group are similar to each other, and different from points in other groups.

★ Everyday picture

Sorting a mixed box of buttons by colour and size, with nobody telling you the category names in advance. You still end up with sensible piles, purely by noticing what looks alike.

Where this shows up in ML. Customer segmentation, anomaly detection, and grouping similar documents or images all start from clustering, with no answer key required.

✗ Watch out

Without labels, there is no single "correct" clustering to check against. Cluster quality has to be judged with internal measures (like the silhouette score, in the next section) or by checking if the groups make sense to a human.

✔ Key takeaway

Unsupervised learning finds groups from similarity alone — there is no teacher's answer key to check against.

13 K-means clustering

☞ The intuition

K-means picks k starting centre points (**centroids**), assigns every data point to its nearest centroid, moves each centroid to the average position of its assigned points, and repeats until nothing changes.

★ Everyday picture

Imagine k ice-cream trucks parked in a park. Every child walks to their nearest truck. Each truck then drives to the average position of the children who came to it. Repeat, and the trucks settle into the natural centres of the crowd.

13.1 The algorithm, worked by hand

✎ Worked example: clustering six points into two groups

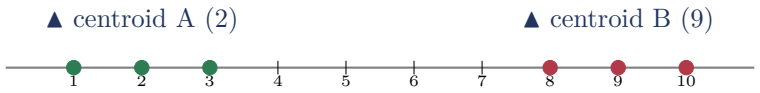
Points on a number line: $1, 2, 3, 8, 9, 10$. We want $k = 2$ clusters. Start with centroids at 1 and 10.

Step 1. Assign each point to its nearest centroid. Distances to centroid 1 vs. centroid 10: point 3 is 2 away from 1 and 7 away from 10, so it joins centroid 1's group. Point 8 is 7 away from 1 and 2 away from 10, so it joins centroid 10's group. This gives groups $\{1, 2, 3\}$ and $\{8, 9, 10\}$.

Step 2. Recompute each centroid as the mean of its group. Mean of $\{1, 2, 3\} = 2$. Mean of $\{8, 9, 10\} = 9$.

Step 3. Reassign with the new centroids 2 and 9. Every point stays in the same group — nothing changed.

Step 4. Stop. The assignment is stable, so k-means has **converged**: cluster A $= \{1, 2, 3\}$ (centre 2), cluster B $= \{8, 9, 10\}$ (centre 9).



How to read it: the green points settle around centroid 2, the red points around centroid 9 — two clearly separated clusters, exactly as the worked example found.

13.2 Choosing k: the elbow method

K-means needs you to choose k in advance. The **elbow method** plots the **within-cluster sum of squares** (WCSS — total squared distance from every point to its own centroid) against k, and looks for the point where adding more clusters stops helping much.

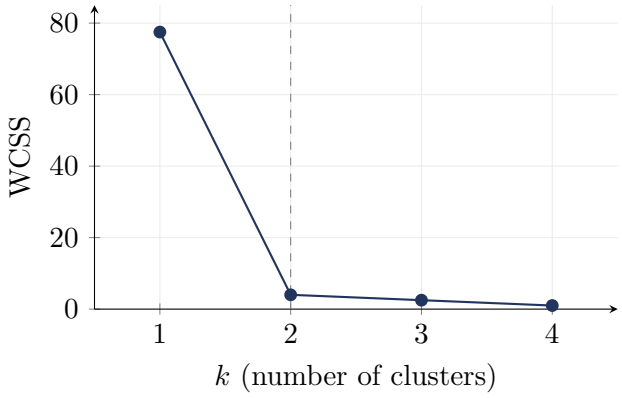
✎ Worked example: finding the elbow for the same six points

Step 1. $k = 1$**: one cluster, mean** $= 5.5$**.** WCSS $= (1-5.5)^2 + (2-5.5)^2 + (3-5.5)^2 + (8-5.5)^2 + (9-5.5)^2 + (10-5.5)^2 = 20.25 + 12.25 + 6.25 + 6.25 + 12.25 + 20.25 =$ **77.5**.

Step 2. $k = 2$**: clusters** $\{1, 2, 3\}$ **and** $\{8, 9, 10\}$**.** Each contributes $(1-2)^2 + (2-2)^2 + (3-2)^2 = 2$. Total WCSS $= 2 + 2 =$ **4.0**.

Step 3. $k = 3$**: split into** $\{1\}, \{2, 3\}, \{8, 9, 10\}$**.** WCSS $= 0 + 0.5 + 2 =$ **2.5**.

Step 4. $k = 4$**: split further into** $\{1\}, \{2, 3\}, \{8, 9\}, \{10\}$**.** WCSS $= 0 + 0.5 + 0.5 + 0 =$ **1.0**.



How to read it: WCSS crashes from 77.5 to 4.0 between k=1 and k=2, then barely improves after that. The sharp bend — the "elbow" — sits at $k = 2$: that is the natural number of clusters here.

13.3 Checking cluster quality: the silhouette score

The **silhouette score** checks, for one point, whether it sits comfortably in its own cluster or is closer to the wrong one:

$$s(x) = \frac{b(x) - a(x)}{\max(a(x), b(x))}$$

$a(x)$ is the average distance from x to other points in its *own* cluster. $b(x)$ is the average distance from x to points in the *nearest other* cluster. $s(x)$ ranges from -1 (badly clustered) to $+1$ (perfectly separated); 0 means the point sits right on the border.

✎ Worked example: scoring the point $x = 3$

Point 3 is in cluster A $= \{1, 2, 3\}$; the nearest other cluster is B $= \{8, 9, 10\}$.

Step 1. Compute $a(3)$. Distance to the other points in cluster A: $|3-1| = 2$, $|3-2| = 1$. Average: $a(3) = (2+1)/2 = 1.5$.

Step 2. Compute $b(3)$. Distance to every point in cluster B: $|3-8| = 5$, $|3-9| = 6$, $|3-10| = 7$. Average: $b(3) = (5+6+7)/3 = 6.0$.

Step 3. Compute $s(3)$. $s(3) = (6.0 - 1.5)/\max(1.5, 6.0) = 4.5/6.0 = \mathbf{0.75}$.

A score of 0.75, close to $+1$, confirms point 3 sits comfortably inside its own cluster, well away from cluster B.

Where this shows up in ML. K-means underlies vector-quantisation and "codebook" methods, and is a standard first step in customer-segmentation pipelines that feed into targeted marketing or recommendation systems.

✗ Watch out

K-means assumes roughly round, similarly sized clusters, and needs k chosen in advance. It struggles badly with long, oddly shaped, or very uneven-sized groups.

✔ Key takeaway

K-means alternates "assign points to nearest centroid" and "move centroid to the mean" until nothing changes; the elbow method and silhouette score help you sanity-check the choice of k.

14 Hands-on labs from this chapter

The lecture pairs each major idea with a lab, using `scikit-learn`. These are worth running yourself alongside this companion:

Activity	Task	Concept practised
Lab 21	House rent prediction (India)	Linear regression
Self Practice 9	Salary prediction	Linear regression
Lab 22	Wine quality classification	Logistic regression / classification
Self Practice 10	Classification on the Framingham heart-study dataset	Classification metrics
Lab 23	Wholesale customer segmentation	K-means clustering

15 Check your understanding

1. **Which method finds the best-fit line in linear regression?** Least squares error — it minimises the total squared distance between the line and every point.

2. **What is the typical order of steps in a supervised learning project?** Data collection, data preprocessing, model selection — cleaning always comes before modelling, never after.

3. **The sample data a model learns from is called...?** Training data — the test data is kept aside and never used for learning.

4. **What does Naive Bayes assume about the features?** That they are independent of each other, given the class — the "naive" part of its name.

5. **What kind of problem is logistic regression built for?** Binary classification — despite the word "regression" in its name.

Glossary & symbols

Key terms.

training data	the labelled examples a model learns from.
test data	examples held back, never trained on, used only for a final fair check.
regression	predicting a continuous number.
classification	predicting a category from a fixed list.
residual	the gap between an actual value and a model's prediction.
overfitting	a model that memorised the training data instead of learning the general pattern.
entropy	a measure of how mixed-up a group of labelled examples is, in bits.
information gain	how much a question reduces entropy after splitting a group.
prior / posterior	your belief before seeing evidence, and your updated belief after.
hyperplane	the dividing line (or surface) an SVM draws between classes.
centroid	the current centre point of one cluster in k-means.
WCSS	within-cluster sum of squares: total squared distance from points to their own centroid.

Symbols at a glance.

Symbol	Meaning
E, T, P	experience, task, performance measure (Mitchell's ML definition)
b_0, b_1	the intercept and slope of a regression line
$\hat{y}$	the model's predicted value for y
$\sigma(z)$	the sigmoid function, $1/(1+e^{-z})$; squashes any number into $(0, 1)$
TP, FP, FN, TN	true/false positive/negative counts in a confusion matrix
$H(S)$	the entropy (mixed-upness, in bits) of a group S
w, b	the weight vector and bias of an SVM hyperplane
$\|w\|$	the length (norm) of the weight vector w
k	the number of clusters chosen for k-means
$s(x)$	the silhouette score of point x, from -1 to $+1$

Further reading

- [databasetown.com](https://databasetown.com/supervised-learning-algorithms/)
- [databasetown.com](https://databasetown.com/unsupervised-learning-types-applications/)
- [mlnotesblog.wordpress.com](https://mlnotesblog.wordpress.com/2017/11/08/decision-tree-entropy-and-information-gain/)
- [data-flair.training](https://data-flair.training/blogs/svm-support-vector-machine-tutorial/)
- [simplilearn.com](https://www.simplilearn.com/tutorials/artificial-intelligence-tutorial/what-is-artificial-intelligence)